

University of Pisa

Parallel and Distributed Systems

Optimization of Partitioned Hash Join with Duplicates

Module 3 - Report

Gianluca Panzani

May 14, 2026

Contents

1	Implementation	1
1.1	Executable variants	1
1.2	Workflow	1
1.3	Parameters	1
2	Parallelization Strategy	2
2.1	Partition phase	2
2.2	Join phase	2
3	Results Analysis	3
3.1	Uniform dataset	3
3.2	Skewed datasets	4
4	Scaling analysis	5
4.1	Weak scalability	5
4.2	Strong scalability	5
5	Correctness Validation	6
6	Conclusions	6

1 Implementation

1.1 Executable variants

The repository contains one sequential baseline and six OpenMP executables which are compared each other. There are three kind of OpenMP families based on the type of parallelization strategy: `loop`, `task` and `taskloop`. The last one is added only as an experimental comparison, to see whether OpenMP’s taskloop construct behaves closer to manual tasks or to ordinary loop scheduling. Each family also has a `_wb` version that adds workload balancing on the join phase. All versions use the same flat open-addressing count table in the local build/probe phase, so the comparison is mostly about the parallel execution model and scheduling policy, not about a different join algorithm. So at the end we have the following six executables: `hashjoin_seq`, `hashjoin_omp_loop`, `hashjoin_omp_loop_wb`, `hashjoin_omp_task`, `hashjoin_omp_task_wb`, `hashjoin_omp_taskloop` and `hashjoin_omp_taskloop_wb`.

1.2 Workflow

The implementation workflow strategy is the following:

- implement the OpenMP variants,
- run correctness checks,
- execute an initial larger grid search for each OpenMP family with a uniform dataset,
- analyze the generated CSV files with the statistics notebooks,
- reduce the grid to the most promising parameter regions,
- run the reduced campaign on uniform and skewed datasets to analyze how the different parallelization strategies perform on the different datasets.

1.3 Parameters

The benchmark scripts run Slurm jobs on one node of the `spmcluster` and append one CSV row per configuration in `src/results/`. The fixed parameters of the main campaign are $N_R = N_S = 50\,000\,000$, $P = 256$, `seed=13`, `max_key=1\,000\,000` and block size 32768 in the reduced grid.

The parameters of the final grids were not chosen directly. Initially i’ve created a larger grid for each family type and then, analyzing the relative notebook files, i’ve chosen the most promising values because of the huge amount of combinations. The grid of the `loop` family explores OpenMP schedules (`static`, `dynamic`, `guided`), chunk sizes, partition/join thread counts and partition block sizes. The grid of the `task` one explores the number of input blocks per partition task, the number of partitions per join task, offset-task granularity and thread counts. And the grid of the experimental `taskloop` family explores the same decomposition through `taskloop` grainsizes.

The reduced grid used for the final results keeps the most stable choices found by those statistics files: guided scheduling for loop variants, explicit task grains in the range 2–4 for the task variants, taskloop grains in the range 2–8 for the experimental taskloop variants, 32 join threads for each family, and 32 or 64 partition threads.

Three datasets are measured. The uniform dataset is the same pseudo-random generation scheme used in the previous module. The skewed datasets are generated with the format `skewed_R.H` with: $R\%$ percentage of records sampled from keys that map to the first $H\%$ of partitions. In the final grids i’ve used `skewed_90.5` and `skewed_90.1`, which concentrate 90% of the records respectively into about 13 and 3 hot partitions.

2 Parallelization Strategy

2.1 Partition phase

In the loop versions, the available work is exposed as regular OpenMP loops and the runtime scheduling policy decides how iterations are assigned to threads. This keeps the code compact and makes schedule tuning simple: the same executable can test `static`, `dynamic` and `guided` behavior through `schedule(runtime)`. In practice this strategy is well suited when the work units are regular and numerous, because the OpenMP loop scheduler has low overhead and the tuning space is mostly reduced to schedule, chunk size and thread count.

The task versions use a more explicit decomposition. Instead of relying only on loop iteration scheduling, they create OpenMP tasks over groups of blocks and partition ranges. This gives more control over the granularity of the work submitted to the runtime, but it also introduces a more delicate tradeoff: small tasks improve balancing opportunities but increase task creation overhead, while large tasks reduce overhead but behave closer to coarse loop chunks. For this reason the task grid includes task-grain parameters in addition to the thread counts.

The taskloop version preserves the loop-shaped formulation but asks OpenMP to generate tasks from it, using `taskloop grainsize(...)`. It is simpler than manually creating explicit tasks and, in the results, often behaves competitively; however, the comparison on which we are focusing on remains between the sequential version and the loop/task strategies.

2.2 Join phase

After partitioning, the join phase is naturally parallel because partition p of R only has to be joined with partition p of S . In other words, for each pair of partitions the join is independent.

The loop versions express this with an OpenMP `for` loop over the partition ids and combine the final `join_count`, `checksum1` and `checksum2` with an OpenMP `reduction`. The task versions instead create explicit OpenMP `tasks` for groups of partition joins and each task writes its result into a private `partial` slot (while the global result is computed after `taskwait`).

The important distinction is between the non-`_wb` and the `_wb` executables. In the non-`_wb` versions, the join work is submitted to OpenMP in the natural partition-id order $0, \dots, P-1$. This is simple and works well when the dataset is uniform, because partitions have similar sizes and each local join has roughly similar cost.

With skewed datasets, however, some partitions contain many more records and duplicate matches. Then a raw partition-id order can create a tail effect: most threads finish their small partitions while some few threads remain busy on hot partitions. The `_wb` variants address this by keeping the same local hash-join code and the same final reduction, but changing the work list given to OpenMP. Before the join starts, partitions are sorted by estimated work,

$$|R_p| + |S_p|,$$

from largest to smallest. The loop `_wb` version uses `schedule(dynamic,1)` on this ordered list, so threads repeatedly take the next heaviest remaining partition. The task `_wb` version creates tasks over the ordered list instead of over raw partition ids. Thus `_wb` means workload-balanced scheduling, not a different join algorithm: it exposes expensive partitions earlier and reduces tail imbalance. It still cannot split one single hot partition across multiple threads, which is why the strongest skew remains difficult.

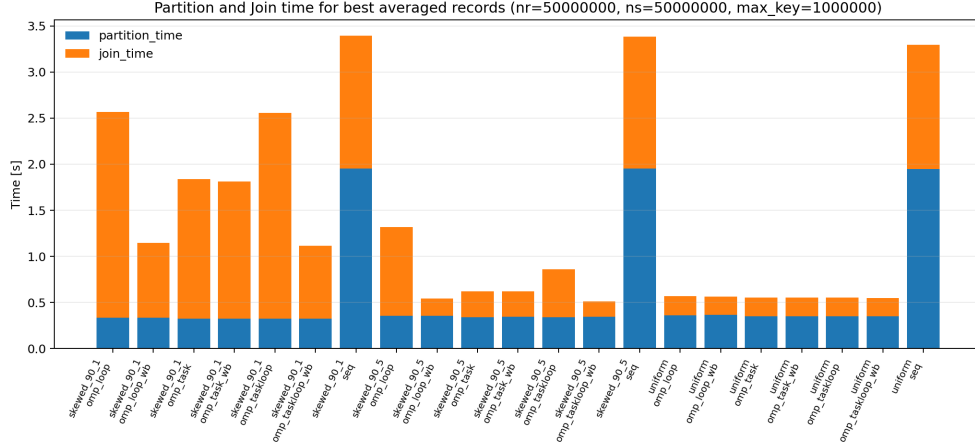


Figure 1: Comparison between partition and join times for each dataset type and executables combination.

3 Results Analysis

All results below are generated from `src/results/`. The best sequential times in the current campaign are 3.114 s for the uniform dataset, 3.258 s for `skewed_90_5`, and 3.256 s for `skewed_90_1`. The best Module 2 C++ threaded result available in `src/old.results/` was 0.507 s for `hashjoin_par_pj_wb_map`; the best broad-grid OpenMP result here is 0.539 s, so the final OpenMP implementation is close in absolute time but does not clearly improve over the previous C++ threaded best case. This covers the main Module 3 comparison: sequential baseline, OpenMP loop, OpenMP tasks, the extra taskloop experiment, uniform/skewed workloads, and the previous C++ threaded implementation.

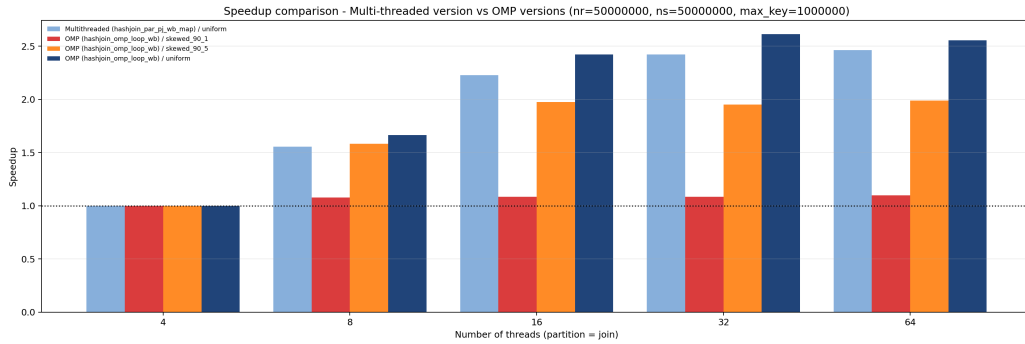


Figure 2: Speedup comparison between the previous multi-threaded `hashjoin_par_pj_wb_map` executable and the selected OpenMP workload-balanced loop implementation.

3.1 Uniform dataset

On the uniform workload, all complete OpenMP implementations are close once both partition and join are parallelized. The best time is obtained by `hashjoin_omp_taskloop_wb` with 64 partition threads and 32 join threads: 0.539 s, corresponding to $5.78\times$ speedup over the current sequential baseline.

Variant	T_{part}	T_{join}	Best time [s]	Speedup
omp_loop	32	32	0.568	5.49×
omp_loop_wb	32	32	0.561	5.55×
omp_task	64	32	0.545	5.72×
omp_task_wb	64	32	0.542	5.74×
omp_taskloop	64	32	0.544	5.73×
omp_taskloop_wb	64	32	0.539	5.78×

Table 1: Best observed uniform-dataset configuration per OpenMP executable.

The difference between the best loop, task and taskloop versions is small; this means that, for uniform partitions, OpenMP tasking does not add much by itself. The dominant gains come from parallelizing the join and from keeping the local hash table cache-friendly. Inside the required implementations, explicit tasks are slightly faster than loop scheduling on uniform data: `omp_task` reaches 0.545 s against 0.568 s for `omp_loop`, and the `_wb` pair has the same small gap. The extra taskloop variant is marginally the best overall in this dataset and is also simpler to express than manual task creation, because OpenMP handles the task generation from the loop structure. The `taskloop` rows are reported only for experimental comparison: the assignment-required comparison is between loop-level OpenMP and explicit tasks.

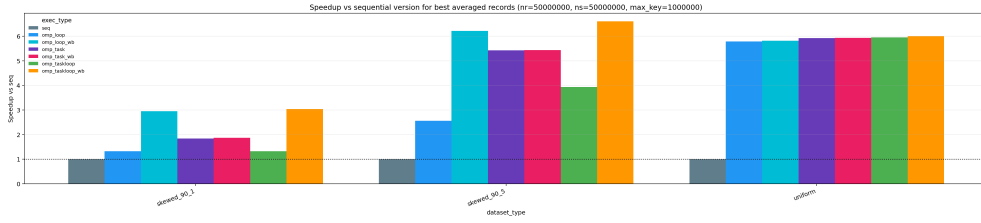


Figure 3: Best speedup against the sequential baseline for the current OpenMP campaign.

3.2 Skewed datasets

The skewed datasets change both load balance and the actual join workload. The final match count rises from about $2.50 \cdot 10^9$ on uniform data to $4.06 \cdot 10^{10}$ on `skewed_90_5` and $1.74 \cdot 10^{11}$ on `skewed_90_1`. Therefore the slowdown is not only scheduling overhead: the hot partitions also contain many more duplicate matches.

Dataset	Variant	T_{part}	T_{join}	Best time [s]	Join time [s]	Speedup
skewed_90_5	omp_loop	32	32	1.281	0.924	2.54×
skewed_90_5	omp_loop_wb	32	32	0.525	0.172	6.20×
skewed_90_5	omp_task	64	32	0.614	0.277	5.31×
skewed_90_5	omp_task_wb	64	32	0.614	0.278	5.30×
skewed_90_5	omp_taskloop	64	32	0.840	0.501	3.88×
skewed_90_5	omp_taskloop_wb	64	32	0.501	0.163	6.51×
skewed_90_1	omp_loop	32	32	2.558	2.222	1.27×
skewed_90_1	omp_loop_wb	32	32	1.120	0.783	2.91×
skewed_90_1	omp_task	64	32	1.802	1.484	1.81×
skewed_90_1	omp_task_wb	64	32	1.799	1.480	1.81×
skewed_90_1	omp_taskloop	64	32	2.539	2.222	1.28×
skewed_90_1	omp_taskloop_wb	64	32	1.101	0.781	2.96×

Table 2: Best observed skewed-dataset result for each OpenMP executable, using the corresponding sequential baseline.

The workload-balanced versions are essential under skew. For `skewed_90_5`, the loop version improves from 1.281 s to 0.525 s when workload balancing is enabled. For `skewed_90_1`, the same

change improves from 2.558 s to 1.120 s. The strongest skew still remains slower because a very small number of partitions becomes too large to be split further by the current partition-level join strategy.

The comparison between the required OpenMP families changes with skew. Loop scheduling performs better than explicit tasks on the skewed datasets once the join work list is balanced: `omp_loop_wb` reaches 0.525 s and 1.120 s, while `omp_task_wb` remains at 0.614 s and 1.799 s. The extra taskloop implementation is still useful experimentally: with workload balancing it gives the best skewed times, while keeping the implementation closer to the loop version than to the more verbose explicit-task code.

4 Scaling analysis

Strong and weak scalability are measured only with the selected workload-balanced loop implementation, using fixed best parameters from the reduced grid. These plots should therefore be read as scalability of the chosen OpenMP loop solution, not as another comparison among loop, task and taskloop implementations. The loop version is used because it is the selected required implementation for the fixed-parameter scaling drivers; the taskloop version is excluded because it is only an additional experimental comparison.

The `strong_scaling.cpp` and `weak_scaling.cpp` files are dedicated scaling drivers. Both keep the same algorithm and the same workload-balanced join scheduling used in the final loop executable: guided partition scheduling, workload-ordered dynamic join scheduling, and fixed block/chunk parameters selected from the reduced grid. The strong-scaling driver fixes $N_R = N_S = 50\,000\,000$ and varies the thread count. The weak-scaling driver increases the input from 10M to 80M records per relation while increasing the thread count from 4 to 32.

4.1 Weak scalability

Weak scaling is moderate. Uniform data keeps the best scaled speedup, reaching about $2.20\times$ at 32 threads relative to the 4-thread base point, but efficiency still drops to about 0.28. The `skewed_90_1` curve is much worse and reaches only $0.89\times$ scaled speedup at 32 threads, because the enlarged hot partitions increase both imbalance and duplicate-match work.

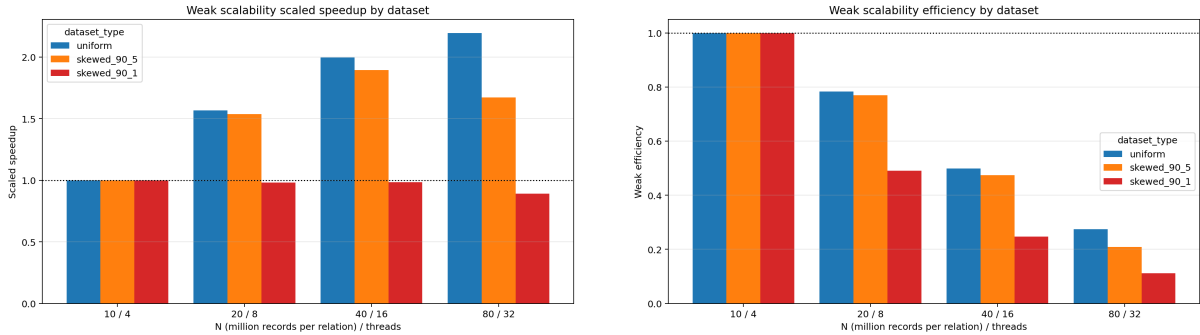


Figure 4: Weak-scaling scaled speedup and efficiency for uniform and skewed datasets.

4.2 Strong scalability

On the uniform dataset, strong-scaling speedup grows up to $6.34\times$ at 32 threads and then slightly decreases at 64 threads. The `skewed_90_5` dataset follows the same shape but with a lower plateau, while `skewed_90_1` saturates very early around $2.9\times$. This confirms that the current granularity is the partition: if one hot partition dominates, adding more threads cannot split that single local join.

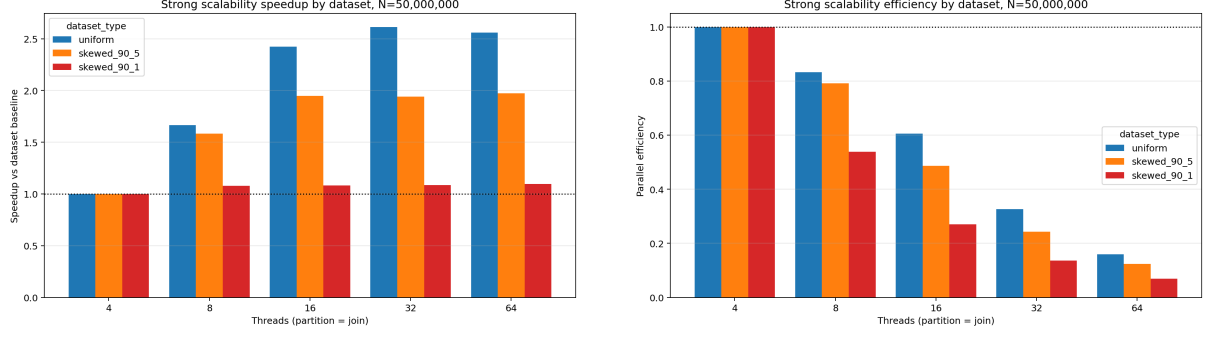


Figure 5: Strong-scaling speedup and efficiency for the dedicated workload-balanced OpenMP executable.

5 Correctness Validation

Correctness is checked at two levels. For small inputs, each executable can run the naive $O(N_R N_S)$ verifier and print `naive_join_count`, `naive_checksum1` and `naive_checksum2`. For benchmark-size inputs, `checker.cpp` scans the Slurm output files and compares `join_count`, `checksum1` and `checksum2` for each executable and dataset type. The checker is outside the measured region, and the collected runs match across implementations for the same dataset.

6 Conclusions

The OpenMP implementations preserve the previous algorithm and make the scheduling trade-offs clearer. On uniform data, loop, task and taskloop variants perform similarly once the join phase is parallelized; tasking does not provide a large advantage because partitions are already well balanced. On skewed data, workload-aware join ordering is the decisive change, especially for `skewed_90_5`. The strongest skew still exposes a limitation of partition-level parallelism: one hot partition cannot be split among multiple threads by the current local join design.

Compared with the best C++ threaded result from Module 2, the best OpenMP result is close but slightly slower in the broad campaign. The main benefit of this module is therefore not a new absolute minimum, but a clearer understanding of which OpenMP constructs are sufficient: guided loop scheduling plus workload-aware dynamic join scheduling is the most robust choice for this implementation.